

# Code Mobility

## Distributed Systems / Case Study

Andrea Omicini

<mailto:andrea.omicini@unibo.it>

Dipartimento di Informatica – Scienza e Ingegneria (DISI)  
ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



- 1 Premise
- 2 Code Mobility



# Next in Line...

1 Premise

2 Code Mobility



# Disclaimer

- most of the following content comes from a well-known article<sup>[Fuggetta et al., 1998]</sup>
- every problem or mistake contained in these slides, however, should be attributed to the sole responsibility of this course's professor



# Next in Line...

1 Premise

2 Code Mobility



# Moving Code

## Sometimes passing data is not enough

- sometimes we would like to change the place where the code is executed—for load balancing, security, scalability, . . .
  - sometimes we do not like to separate the data from the code to be executed on them (e.g., objects, agents)
- then, passing data between processes is no longer enough
- code should be passed

# Reasons for Migrating Code

## Process migration

- traditionally, code is moved along with the whole computational context
- moving code is typically moving processes <sup>[Milojicic et al., 2000]</sup>

## Why?

- load balancing
- minimising communication
- optimising perceived performance
- improving scalability
- flexibility through dynamic configurability
- improving fault tolerance

# Understanding Code Mobility<sup>[Fuggetta et al., 1998]</sup> |

There is much more than just moving code

- what do we move along with a program?
- execution status, pending signals, data, ...

What is the essence of a process?

- it has a set of **actions** to execute
- it has a **state**
- it works within an **environment**, a (computational) *context*



# Understanding Code Mobility<sup>[Fuggetta et al., 1998]</sup> II

## A process model for code migration

- a process can be thought as three segments
  - code segment** the set of the executable instructions of the process
  - execution segment** the store for the execution state of the process—with private data, stack, program counter
  - resource segment** the set of the references to the external resources needed by the process—like files, printers, devices, other processes, . . .
- depending on *which portion* of a process is moved along with the code, we can classify different types of code mobility

# Weak Mobility

## The bare minimum for code migration

- only the code segment is transferred
- possibly along with some initialisation data

## Main idea

- the code can be executed every time *ex novo*
- so, we do not care about any computational context
- or, maybe, the computational context we need is the target one

## Main benefit

- the only requirement is that the target machine can execute the code
- weak mobility is very simple
- it has no particular restrictions or further requirements to be implemented

# Strong Mobility

## Moving execution context

- the execution segment is transferred along with the code segment

## Main benefit

- a process can be stopped, moved, and then restart on another machine

## Requirements

- strong mobility is very demanding
- the technological environment – typically, the middleware – should allow for it

# Sender- vs. Receiver- Initiated Migration I

## Sender-initiated migration

- migration is initiated where the code resides / is being currently executed
  - examples: search-bots, mobile agents
  - servers should know clients, and ensure security of resources
- more complex interaction scheme



# Sender- vs. Receiver- Initiated Migration II

## Client-initiated migration

- migration is initiated by the target machine, requiring a new behaviour to be added
  - examples: Java Applets, JavaScript chunks
  - just a few resources on clients need to be secured
  - clients may also be anonymous
- less complex interaction scheme

# Separate vs. Target Process Execution

## Weak mobility and execution of mobile code

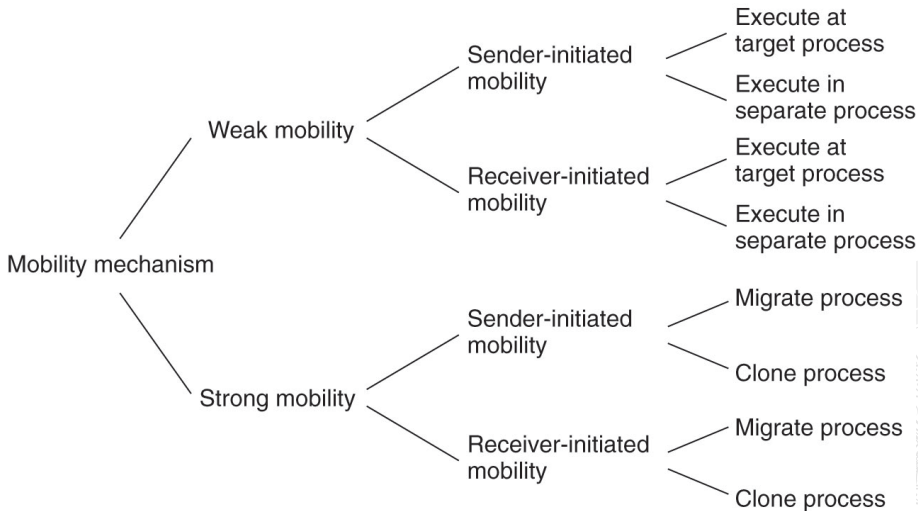
- in the case of weak mobility, one may execute the mobile code on either the target process or a separate process
- for instance, Java Applets are executed in the browser's address space
- no need for inter-process communication at the target machine
- main problem: protection against malicious or buggy code execution
- solution: assigning mobile code execution to a separated process

# Cloning vs. Migrating

## Strong mobility can be supported also by remote cloning

- **cloning** yields an exact copy of the original process, executed on the target machine
- cloned process is executed in parallel to the original process, on different machines
- example: In UNIX, forking a child process and let it execute on a remote machine
- cloning is an alternative to migration
- cloning in some sense improve distribution transparency, in that the processes are transparently replicated on many different machines

# Models for Code Migration



Alternatives for code migration [Tanenbaum and van Steen, 2017]



# Migration and Local Resources

## Migration and the resource segment

- till now, we have only accounted for migration of the code and execution segments
- main problem: resources might not be as easy to move around as code and variables
- example: a huge database might in theory be moved across the network, but in practice it will not
- either references need to be updated, or resources need to be moved

## Two issues

- how does the resource segment refer to resources?
- how does the resource relate with the hosting machine?

# How does the Resource Segment Refer to Resources?

## Process-to-resource binding

**binding by identifier** need of a resource with a given name—e.g., via an URL, or a local ID

**binding by value** need of a resource based on its value—e.g., code libraries

**binding by type** need of a resource based on its type—typically, local devices like printers, monitors, . . .



# How does the Resource Relate with the Hosting Machine?

## Resource-to-machine binding

**unattached resources** resources that can be easily moved between different machines—like, files associated to the migrating code

**fastened resources** resources that can be moved, but at a cost—like, a local database

**fixed resources** resources bounded to a specific machine—like, a monitor



# Code Migration and Local Resources

|                             |               | Resource-to-machine binding |               |            |
|-----------------------------|---------------|-----------------------------|---------------|------------|
| Process-to-resource binding |               | Unattached                  | Fastened      | Fixed      |
|                             | By identifier | MV (or GR)                  | GR (or MV)    | GR         |
|                             | By value      | CP (or MV,GR)               | GR (or CP)    | GR         |
|                             | By type       | RB (or MV,CP)               | RB (or GR,CP) | RB (or GR) |

GR    Establish a global systemwide reference  
 MV    Move the resource  
 CP    Copy the value of the resource  
 RB    Rebind process to locally-available resource

Actions to be taken with respect to the references to local resources when migrating code to another machine

[Tanenbaum and van Steen, 2017]

# Summing Up

## Code migration

- code may move through distributed machines for a number of good reasons
- different types of code mobility are possible, depending on either the application needs or the technology constraints



# Code Mobility

## Distributed Systems / Case Study

Andrea Omicini

<mailto:andrea.omicini@unibo.it>

Dipartimento di Informatica – Scienza e Ingegneria (DISI)  
ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



# References

- [Fuggetta et al., 1998] Fuggetta, A., Picco, G. P., and Vigna, G. (1998).  
Understanding code mobility.  
*IEEE Transactions on Software Engineering*, 24(5):342–361  
(APiCe) DOI:10.1109/32.685258
- [Milojicic et al., 2000] Milojicic, D. S., Douglass, F., Paindaveine, Y., Wheeler, R., and Zhou, S. (2000).  
Process migration.  
*ACM Computing Surveys*, 32(3):241–299  
(APiCe) DOI:10.1145/367701.367728
- [Tanenbaum and van Steen, 2017] Tanenbaum, A. S. and van Steen, M. (2017).  
*Distributed Systems. Principles and Paradigms*.  
Pearson Prentice Hall, 3rd edition  
<https://www.distributed-systems.net/index.php/books/ds3/>

